

# FAILURE MEMORIES ARE NOT NEUTRAL: CAUSAL PROVENANCE EFFECTS IN SELF-EVOLVING AGENTS

**Anonymous authors**

Paper under double-blind review

## ABSTRACT

Self-evolving agents convert previous trajectories into reusable textual memory. Existing analyses usually treat the retrieved memory as the relevant state variable. We study a second variable carried into that state: the provenance of the trajectory from which the memory was written. A recent financial-agent audit reports a sharp provenance-conditioned split for ReasoningBank. Success-derived retrievals achieve 0.931 accuracy over 101 cases. Failure-derived retrievals achieve 0.647 accuracy over 34 cases. All ten reported persistent  $W \rightarrow W$  failures occur in the failure-derived stratum. We show that this pattern is compatible with a direct causal channel because ReasoningBank writes successful trajectories and failed trajectories through different reflection modes before future retrieval. We formulate memory provenance as an intervention variable and separate its causal effect from task difficulty through a matched-memory swap. The experiment holds the future query and semantic guidance fixed while replacing failure-derived memory with a success-derived counterpart. The reverse swap provides the complementary intervention. This design tests whether failure provenance itself sustains future error. Our source-grounded decomposition also reveals that the strongest observed provenance signal concerns persistence: most failures inside the failure-derived stratum are  $W \rightarrow W$  cases. This motivates provenance-aware memory governance for self-evolving agents. A memory should carry information about how it was learned because semantically similar advice can encode distinct process evidence when it originates from success or failure.

## 1 INTRODUCTION

Self-evolving agents increasingly learn through external memory. A successful episode can become a reusable workflow. A failed episode can become a corrective reflection. Both objects may later be retrieved for a new task. Systems such as ReasoningBank make this loop explicit by converting trajectories into reasoning memories that influence future behavior (Ouyang et al., 2026). Related systems store workflows or verbal feedback as persistent state (Wang et al., 2024; Shinn et al., 2023).

The usual abstraction treats memory content as the object that matters. This hides a second variable: how the memory was produced. A memory induced after success carries evidence that its reasoning process reached a valid outcome. A memory induced after failure carries evidence that the original process broke somewhere. ReasoningBank reflects this distinction in its implementation. Successful trajectories are summarized under a success-analysis objective. Failed trajectories are processed under a failure-analysis objective. The resulting memories can contain similar procedural advice while encoding different causal interpretations of the same task family.

A recent audit of self-evolving financial agents provides a strong empirical signal for this distinction (Li & Zhu, 2026). In its ReasoningBank analysis, success-derived retrievals have reported accuracy 0.931 over 101 cases. Failure-derived retrievals have reported accuracy 0.647 over 34 cases. The audit further reports ten persistent  $W \rightarrow W$  failures, where a task fails before evolution and continues to fail afterward. Every one of these persistent failures appears in the failure-derived retrieval stratum. Figure 2 summarizes the released counts.

This association points to a causal question with direct implications for memory governance: does provenance itself change future behavior after semantic guidance is controlled? Task difficulty provides the strongest competing explanation. Hard tasks are more likely to fail during acquisition.

Failure provenance is an upstream intervention on memory construction

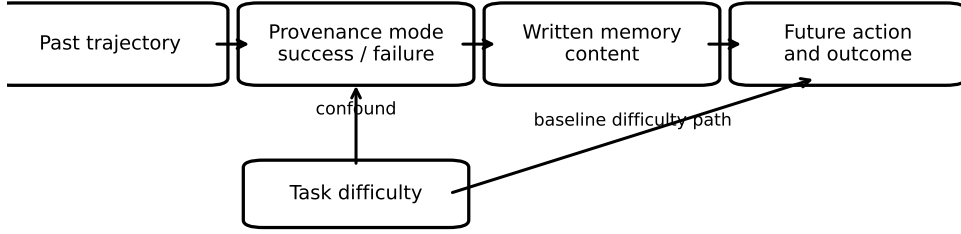


Figure 1: Causal structure of provenance-conditioned memory. Task difficulty affects which trajectories fail and also affects future outcomes. The matched-memory intervention blocks this confound by fixing the future query and matching semantic guidance across provenance conditions.

Those same tasks can later retrieve memories derived from failure. A descriptive provenance gap can therefore emerge even when provenance has no causal effect. We resolve this identification problem with a matched provenance intervention.

The intervention begins with a future query and a retrieved failure-derived memory. We construct a success-derived counterpart that matches the guidance at the semantic level. The future query remains fixed. The agent scaffold remains fixed. Replacing only the memory provenance creates the causal contrast. A reverse swap tests the opposite direction. The resulting estimand measures whether failure-derived memory sustains error beyond the information carried by the memory itself.

The paper makes three contributions through separate evidence layers. First, we reanalyze the released financial-agent audit and show that the strongest provenance-conditioned signal is persistent failure. Second, we formalize provenance as an upstream state variable in memory construction. Third, we specify a matched-memory causal design that isolates the effect of provenance from task difficulty. These results motivate a provenance-aware representation for self-evolving memory systems. The source of a memory becomes part of the state that governs whether the agent should reuse it.

## 2 MEMORY PROVENANCE AS PERSISTENT AGENT STATE

Consider an acquisition episode with task  $x$  and trajectory  $\tau$ . Its terminal outcome is  $z \in \{0, 1\}$ . A memory writer receives the episode and produces a textual memory

$$M = G(\tau, z). \quad (1)$$

ReasoningBank instantiates  $G$  with outcome-conditioned reflection. The successful branch asks the writer to explain the strategy that worked. The failure branch asks the writer to identify the cause of failure and construct an improved approach. We call the branch variable  $P \in \{S, F\}$  the memory provenance.

A future query  $x'$  retrieves a memory through a similarity function  $R(x', x)$ . The retrieval key can be identical even when the written memory differs because the key is the historical task description. Future behavior then follows

$$Y = H(x', M(P), S), \quad (2)$$

where  $S$  denotes the rest of the agent state. Provenance reaches future behavior through memory content. It is therefore an upstream causal variable with direct influence on the persistent state.

### 2.1 TARGET ESTIMAND

Let  $C(M)$  denote the semantic guidance contained in a memory. The quantity of interest is the effect of provenance under matched guidance:

$$\Delta_P(x') = \mathbb{E}[Y \mid do(P = F), C(M) = c, x'] - \mathbb{E}[Y \mid do(P = S), C(M) = c, x']. \quad (3)$$

Table 1: Provenance-conditioned ReasoningBank outcomes from the released financial-agent audit.

| Retrieved memory provenance | Cases | Correct | Accuracy |
|-----------------------------|-------|---------|----------|
| Success-derived             | 101   | 94      | 0.931    |
| Failure-derived             | 34    | 22      | 0.647    |

For a success indicator, a negative  $\Delta_P$  means failure provenance reduces future success under matched information. For an error indicator, the sign reverses.

Task difficulty  $D$  creates the central identification challenge. Difficult acquisition tasks yield more failed trajectories. They also remain difficult when encountered later. Conditioning only on observed provenance therefore estimates a mixture of provenance effect and task difficulty. Equation 3 motivates a direct intervention on the retrieved memory while the future task remains fixed.

## 2.2 MATCHED PROVENANCE PAIRS

A provenance pair contains two memories with the same intended procedural guidance. One member is derived from a successful trajectory. The other member is derived from a failed trajectory. Matching is performed over the task family and the actionable recommendation. Each pair is checked for information asymmetry before it enters the experiment. A pair is rejected when one memory contains an extra fact that can independently solve the future task.

This construction turns provenance into an experimentally manipulable state variable. It also exposes a design principle for memory systems: a textual memory is incomplete as an audit object when its learning process has been discarded. Provenance metadata preserves evidence about whether the remembered strategy emerged from demonstrated success or from a corrective interpretation of failure.

## 3 RELEASED EVIDENCE FOR A PROVENANCE EFFECT

We begin from the public aggregate audit of ReasoningBank on benign AgentDojo Banking endpoints (Li & Zhu, 2026). The audit separates retrievals according to whether the retrieved memory was derived from a successful episode or from a failed episode. Table 1 reproduces the reported counts after converting the rounded accuracies back to their integer support.

The observed accuracy gap is 28.4 percentage points. This is a large effect at the retrieval stratum level. The transition audit gives the provenance signal a sharper interpretation. ReasoningBank records 25  $W \rightarrow C$  recoveries and 9  $C \rightarrow W$  regressions. Ten  $W \rightarrow W$  persistent failures remain after evolution. All ten belong to the failure-derived retrieval stratum.

The rounded provenance accuracies imply 12 failures among 34 failure-derived retrievals and 7 failures among 101 success-derived retrievals. The ten  $W \rightarrow W$  cases therefore account for roughly 83.3% of failures in the failure-derived stratum. The success-derived stratum contains no  $W \rightarrow W$  case in the audit. Figure 2 visualizes this decomposition.

### 3.1 THE SIGNAL IS STRONGEST FOR PERSISTENCE

The transition structure changes the scientific interpretation of the aggregate accuracy gap. Failure-derived memory is tightly associated with errors that persist across self-evolution. The source audit identifies task difficulty as a confound for this association. Our causal formulation preserves the persistence signal as the main target and removes task difficulty through an intervention on memory provenance.

A simple arithmetic check also constrains the mechanism. The failure-derived stratum contains roughly two failures beyond the ten  $W \rightarrow W$  cases. The success-derived stratum contains seven failures. The provenance signal therefore concentrates on persistent error much more strongly than on newly induced regression. This pattern motivates a paper claim centered on failure persistence.

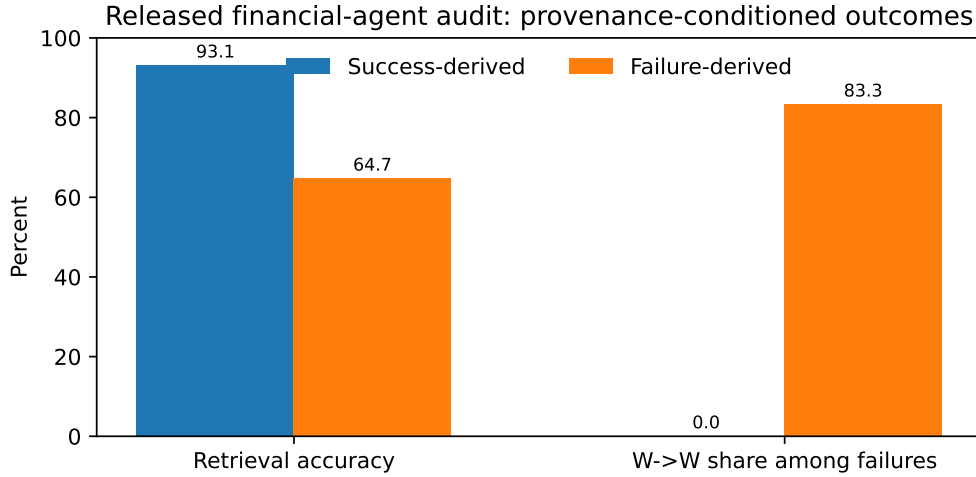


Figure 2: The released audit shows a large accuracy gap by memory provenance. Persistent  $W \rightarrow W$  failures are concentrated in failure-derived retrievals.

### 3.2 INDEPENDENT MECHANISM EVIDENCE

Memory Reward Inflation studies a different memory architecture and identifies an amplification channel for erroneous memories (Asadolahi et al., 2026). Incorrect memories can receive inflated utility scores, which increases their future reuse. The paper also shows that this behavior can persist under similarity-based retrieval. This mechanism is compatible with the provenance effect studied here. A failure-derived memory can carry corrective advice while remaining anchored to a trajectory whose process already demonstrated an error mode. Repeated reuse can then sustain that mode when the corrective abstraction is incomplete.

Together, the financial-agent audit and reward-inflation results support a causal model in which memory provenance changes the distribution of future behavior. The matched provenance experiment in Section 4 provides the direct test.

## 4 CAUSAL PROVENANCE INTERVENTION

The decisive experiment edits the retrieved memory while leaving the future task unchanged. We call this the matched provenance swap.

### 4.1 PAIR CONSTRUCTION

For each future task that retrieves a failure-derived memory, we identify a successful trajectory from the same task family. The two trajectories are converted into memories using the same writer model. Each memory is then compressed to a matched procedural core. A verifier checks whether the pair exposes the same actionable information to the future agent. The pair enters the experiment when the procedural content is matched and the provenance differs.

The intervention produces two conditions. The first condition injects the failure-derived memory. The second condition injects the success-derived counterpart. The agent receives the same future query under both conditions. Model sampling is frozen. Tool state is frozen. The benchmark environment starts from the same snapshot.

### 4.2 REVERSE SWAP

A complementary cohort begins with future tasks that normally retrieve success-derived memory. The intervention replaces that memory with a matched failure-derived counterpart. This direction tests whether failure provenance can introduce persistence into tasks that previously had access to

successful guidance. The forward swap measures recovery from a failure-derived state. The reverse swap measures degradation after the state is introduced.

#### 4.3 OUTCOME MEASURES

The primary endpoint is task success on the matched future query. A secondary endpoint measures action divergence before terminal feedback. We record the first decision point at which the two provenance conditions choose different tool actions or different subgoals. This localizes the mechanism before outcome differences accumulate over a long trajectory.

A provenance effect is supported when matched failure-derived memory produces lower future success or earlier error-consistent action divergence. The effect should replicate across task families. We also report pair-level heterogeneity because some failed trajectories may generate unusually strong corrective memories.

#### 4.4 MEMORY GOVERNANCE IMPLICATION

A positive effect changes how self-evolving memory should be represented. Memory retrieval should preserve provenance as structured metadata. The agent can then condition trust on the demonstrated process behind the memory. A failure-derived item can remain useful while receiving a different reuse policy from a memory backed by successful execution.

This design is compatible with ReasoningBank because the acquisition mode is already explicit during memory writing (Ouyang et al., 2026). It also transfers to workflow memory systems that store experience-derived procedures (Wang et al., 2024). The intervention operates at the memory boundary, so it does not require parameter updates or changes to the base model.

### 5 RELATED WORK

**Experience memory for self-evolving agents.** ReasoningBank turns previous trajectories into reusable reasoning memories and retrieves them for future tasks (Ouyang et al., 2026). Agent Workflow Memory stores reusable task workflows induced from experience (Wang et al., 2024). Reflexion uses verbal feedback as persistent context for later trials (Shinn et al., 2023). These systems establish external textual state as an effective route to self-improvement. Our work adds trajectory provenance as a causal variable inside that state.

**Failure memory and memory quality.** The financial-agent audit reports strong concentration of persistent failure in failure-derived ReasoningBank retrievals (Li & Zhu, 2026). Memory Reward Inflation shows that erroneous memories can gain excessive effective reward and propagate through future reuse (Asadolahi et al., 2026). These findings establish that memory quality affects future behavior. Our intervention isolates a more specific mechanism: the process provenance attached to semantically matched guidance.

**Causal evaluation of memory.** Existing ablations often remove a memory module or compare agents with different memory banks. Those interventions change both information and provenance. The matched provenance swap preserves the actionable guidance seen by the future agent. This creates a same-information causal test for the history encoded in the memory itself.

### 6 LIMITATIONS

The current public financial-agent release provides aggregate provenance counts and transition statistics. The matched provenance swap requires the first-party per-query memory artifacts or a faithful reconstruction of the audited ReasoningBank runs. Those artifacts are announced for release upon publication and are currently experiment debt.

The reported provenance accuracies are rounded to three decimals. The integer correct counts used in our decomposition are the unique counts consistent with the reported support sizes and rounded

accuracies. The transition allocation beyond the ten  $W \rightarrow W$  cases is therefore a derived arithmetic decomposition from public aggregate statistics.

Semantic matching between failure-derived memory and success-derived memory is an experimental design challenge. A pair can preserve procedural content while differing in wording. We therefore require an information-equivalence audit before the swap. Pair construction and verifier agreement remain experiment debt until the first-party memory artifacts are available.

The causal claim concerns future persistence under matched memory information. The existing audit supports the provenance-conditioned association and identifies task difficulty as a confound. Direct causal effect size remains experiment debt for the swap experiment in Section 4.

The empirical source is a financial AgentDojo setting with ReasoningBank. Replication in web agents and other external-memory architectures remains experiment debt for a broader cross-domain claim.

## 7 CONCLUSION

Self-evolving memory carries process history. The released financial-agent audit shows a large accuracy gap by memory provenance and concentrates every reported persistent  $W \rightarrow W$  failure in failure-derived retrievals. ReasoningBank makes provenance an upstream intervention because successful trajectories and failed trajectories are written through different reflection objectives.

We formalize the causal effect of that provenance and define a matched-memory swap that holds the future query and actionable guidance fixed. This turns memory provenance into an experimentally identifiable variable. The resulting perspective suggests a concrete design principle for self-evolving agents: store how a memory was learned and use that evidence when deciding how strongly future behavior should trust it.

## REFERENCES

- Mohammad Asadolahi, Amir Amini, Samira Talebi, Amirfarhad Farhadi, and Azadeh Zamanifar. Memory reward inflation in self-improving llm agents, 2026.
- Jialong Li and Jialing Zhu. Auditing self-evolution in financial agents: Capability gains, security drift, and execution-interface mismatch, 2026.
- Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long T. Le, Samira Daruki, Xiangru Tang, Vishy Tirumalashetty, George Lee, Mahsan Rofouei, Hangfei Lin, Jiawei Han, Chen-Yu Lee, and Tomas Pfister. Reasoningbank: Scaling agent self-evolving with reasoning memory. In *International Conference on Learning Representations*, 2026.
- Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023.
- Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory, 2024.

## A SOURCE-GROUNDED REANALYSIS DETAILS

### A.1 COUNT RECONSTRUCTION

The financial-agent audit reports 101 success-derived retrievals at 0.931 accuracy. The unique integer numerator consistent with this value at three decimal places is 94. It reports 34 failure-derived retrievals at 0.647 accuracy. The corresponding integer numerator is 22. This yields 7 failures in the success-derived stratum and 12 failures in the failure-derived stratum.

The same audit reports ten  $W \rightarrow W$  persistent failures, all with failure-derived retrieval. Hence 10 of the 12 failures in the failure-derived stratum are persistent failures. The remaining two failure-derived errors can contribute to the reported  $C \rightarrow W$  regressions. The success-derived stratum contains no  $W \rightarrow W$  cases and contributes seven failures to the non-persistent error pool.

## A.2 MATCHED-PAIR ACCEPTANCE CONTRACT

A proposed provenance pair must satisfy four checks. The future task is identical under both interventions. The two memory items must support the same procedural objective. One member must come from a successful acquisition trajectory and the other from a failed acquisition trajectory. A verifier must find no extra task-solving fact in either memory. Pairs that fail any check are excluded before outcomes are observed.

## A.3 REOPEN CONDITION

The experiment advances immediately when first-party query-level ReasoningBank retrieval artifacts become available. A faithful reconstruction using the released AgentDojo Banking setup can also satisfy the support requirement when it reproduces the published provenance counts within sampling tolerance.